

TP7 — Générer des configs réseau dynamiques via templates

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Ce TP utilise les variables produites au TP6 (host_vars/<site>.yaml) pour générer, via un template Jinja2, la configuration réseau finale de chaque site. Il introduit aussi l'organisation en dossier playbooks/, réutilisée pour tous les TP suivants, et un fichier ansible.cfg qui simplifie durablement les commandes.

Prérequis

- TP1 à TP6 validés (les fichiers inventory/host_vars/*.yaml du TP6 doivent exister)
- Ansible fonctionnel

Étapes

Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour2-tp7
```

Étape 2 — Réorganiser en dossier playbooks/ (et migrer le TP5)

À partir de maintenant, tous les playbooks vivent dans playbooks/, pas à la racine — sinon la racine du projet finirait polluée par un fichier par TP.

```
mkdir -p playbooks
git mv site.yaml playbooks/site.yaml
```

Étape 3 — Créer ansible.cfg à la racine du projet

```
cat > ansible.cfg << 'EOF'
[defaults]
roles_path = ./roles
```

```
inventory = ./inventory/dynamic_inventory.py
EOF
```

Sans ce fichier, déplacer site.yml dans playbooks/ casse la résolution du rôle base_config (Ansible cherche les rôles relativement au dossier du playbook par défaut) : The role 'base_config' was not found. roles_path dans ansible.cfg corrige ça pour tous les playbooks présents et futurs.

Sous WSL, si vous obtenez le warning Ansible is being run in a world writable directory ... ignoring it as an ansible.cfg source : WSL monte les disques Windows (/mnt/c/...) avec des permissions qu'Ansible considère par sécurité comme "world writable", et il refuse alors de charger ansible.cfg. Forcez son chargement explicitement :

```
export ANSIBLE_CONFIG=/mnt/c/Users/<votre-nom>/Desktop/netops-project/ansible.cfg
```

Rendez-le permanent :

```
echo 'export ANSIBLE_CONFIG=/mnt/c/Users/<votre-nom>/Desktop/netops-project/ansible.cfg' >> ~/.bashrc
source ~/.bashrc
```

Étape 4 — Vérifier que le TP5 fonctionne toujours après la migration

```
cd lab && docker compose up -d --build && cd ..
ansible-playbook playbooks/site.yml
```

Remarquez : plus besoin de -i inventory/dynamic_inventory.py — ansible.cfg le fait automatiquement.

Si l'inventaire ne se charge pas avec une erreur du type env: \$'python3\r': No such file or directory : le fichier inventory/dynamic_inventory.py contient des fins de ligne Windows (CRLF), qui cassent la ligne shebang. Corrigez :

```
sed -i 's/\r$//' inventory/dynamic_inventory.py
chmod +x inventory/dynamic_inventory.py
```

Et pour renormaliser tout le dépôt d'un coup, si le .gitattributes du TP2 existe déjà mais que certains fichiers plus anciens n'en ont pas encore profité :

```
git add --renormalize .
git commit -m "Renormalisation des fins de ligne (gitattributes)"
```

Étape 5 — Ajouter des variables communes à tous les sites

```
cat > inventory/group_vars/all.yml << 'EOF'
---
ntp_server: 10.0.0.1
banner_motd: "Acces reserve au personnel autorise - NexaCorp"
EOF
```

group_vars/all.yml s'applique à **tous** les hôtes, alors que host_vars/<site>.yaml (du TP6) ne s'applique qu'à un site précis — Ansible fusionne automatiquement les deux pour construire le jeu de variables final de chaque hôte. C'est ça, les "variables complexes".

Étape 6 — Écrire le template Jinja2

```
mkdir -p templates
cat > templates/interface_config.j2 << 'EOF'
! Configuration generee automatiquement - NE PAS EDITER A LA MAIN
! Site : {{ inventory_hostname }}
!
hostname {{ inventory_hostname }}
!
{% for iface in interfaces %}
interface {{ iface.name }}
    description {{ iface.description }}
    switchport access vlan {{ iface['vlan-id'] }}
{% if iface.enabled == 'down' %}
    shutdown
{% else %}
    no shutdown
{% endif %}
!
{% endfor %}
ntp server {{ ntp_server }}
banner motd ^ {{ banner_motd }} ^
EOF
```

Étape 7 — Écrire le playbook de génération dans playbooks/

```
cat > playbooks/generate_configs.yml << 'EOF'
---
- name: Generer les configurations reseau a partir des templates
  hosts: all
  gather_facts: no
  tasks:
    - name: Creer le dossier de sortie
      ansible.builtin.file:
        path: "{{ playbook_dir }}/../generated_configs"
        state: directory
        run_once: true
        delegate_to: localhost

    - name: Generer la configuration de l'interface pour ce site
      ansible.builtin.template:
        src: "{{ playbook_dir }}/../templates/interface_config.j2"
```

```
dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
EOF
```

Notez `{{ playbook_dir }}/../templates/...` : comme le playbook est dans `playbooks/`, il faut remonter d'un niveau (`..`) pour retrouver `templates/` et écrire dans `generated_configs/` à la racine du projet.

Étape 8 — Exécuter le playbook

```
ansible-playbook playbooks/generate_configs.yml
cat generated_configs/siege.cfg
```

Résultat attendu :

```
! Configuration generee automatiquement - NE PAS EDITER A LA MAIN
! Site : siege
!
hostname siege
!
interface GigabitEthernet0/1
  description Lien vers coeur de reseau
  switchport access vlan 10
  no shutdown
!
interface GigabitEthernet0/2
  description Reserve
  switchport access vlan 20
  shutdown
!
ntp server 10.0.0.1
banner motd ^ Acces reserve au personnel autorise - NexaCorp ^
```

Étape 9 — Vérifier l'idempotence

Relancez exactement la même commande, sans rien changer :

```
ansible-playbook playbooks/generate_configs.yml
```

Résultat attendu : ok partout, changed=0.

Étape 10 — Vérifier qu'un vrai changement est bien détecté

```
sed -i 's/Lien vers coeur de reseau/Lien vers coeur de reseau - MAJ/' inventory/host_vars/siege.yml
ansible-playbook playbooks/generate_configs.yml --limit siege
```

Résultat attendu : changed=1 sur siege uniquement.

Étape 11 — Committer et pousser

```
echo "generated_configs/" >> .gitignore
git add ansible.cfg playbooks/ templates/ inventory/group_vars/ inventory/dynamic_inventory.py .gitignore
```

```
git commit -m "TP7 : reorganisation playbooks/, ansible.cfg, templates Jinja2, variables complexes"
git push -u origin <prenom>-<nom>-jour2-tp7
```

Ouvrir une pull request <prenom>-<nom>-jour2-tp7 → dev.

Comprendre le TP

Un template Jinja2 ressemble à la sortie finale, avec des zones variables (`{{ ... }}`) et de la logique (`{% for %}`, `{% if %}`). Le module template d'Ansible remplace chaque variable par sa valeur réelle (venue de `host_vars`, `group_vars`, ou des deux combinés) et écrit le résultat. La boucle parcourt la liste d'interfaces produite au TP6, sans avoir à écrire une ligne de configuration à la main pour chaque interface de chaque site.

L'idempotence est ce qui rend ce mécanisme sûr à ré-exécuter : Ansible compare le contenu qu'il va écrire avec ce qui existe déjà, et ne touche au fichier que si une différence réelle existe.

Le passage à `playbooks/` + `ansible.cfg` n'est pas qu'une question de rangement : ça évite qu'un projet qui va accumuler un playbook par TP devienne illisible, et ça centralise des réglages (inventaire, chemin des rôles) qu'on aurait sinon dû répéter en argument à chaque commande.

Validation

- ☐ `site.yml` (TP5) est déplacé dans `playbooks/` et fonctionne toujours grâce à `ansible.cfg`
- ☐ `ansible-playbook playbooks/generate_configs.yml` fonctionne sans `-i` (grâce à `ansible.cfg`)
- ☐ `generated_configs/siege.cfg` contient bien les interfaces du TP6 et les variables communes
- ☐ Une deuxième exécution sans changement affiche `changed=0` partout
- ☐ Une modification d'une variable `host_vars` fait réapparaître `changed=1` uniquement sur le site concerné
- ☐ `generated_configs/` est ignoré par Git
- ☐ Une pull request <prenom>-<nom>-jour2-tp7 → dev est ouverte